

# 二諦随伴上の帰納モナド： 論理体系からプログラミング言語へ

Masaomi Chiba

2026-02-25

## Abstract

本稿では、任意の論理体系または文法体系  $N$  を実行可能なプログラミング言語へと変換する普遍的機構として帰納モナド  $I_N$  を導入する。ここでいう「弱い」とは、論理体系がそのままではプログラミング言語としての実行構造（制御・状態・探索など）を備えていないという意味であり、欠陥ではなく設計上の理由で生じる「隙間」によるものである。その隙間をモナド構造が自然に埋めるという観察が鍵である。日常的視点 ( $C/Dfix0$ ) と究極的視点 ( $C^D$ ) の間にある二諦随伴  $F_n \dashv G_{2^n+1}$  を用いることで、任意の体系  $N$  に適切な帰納モナドを付与すればチューリング完全（あるいは計算的に意味のある）言語が得られることを示す。論理・計算・文法にまたがる4つの典型例（ペアノ算術・最小論理・文脈自由文法・ホーン節）を示し、10の基本体系と最適なモナドの対応表を与える。本枠組みはプログラム設計原理を統一的に説明し、「表現力の隙間」が偶然ではなく欠落したモナドにより構造的に決定されることを形式的に裏付ける。

## 1 序論

### 1.1 根本的な隙間

あらゆる形式体系には記述と実行の間に固有の隙間が存在する：

- ペアノ算術は加法を記述できるが自由に計算できない。
- 最小論理は含意を表現できるが否定や制御構造を持たない。
- 文脈自由文法は構文を定義できるが解析状態を管理できない。
- ホーン節は関係を記述できるが明示的なバックトラック探索を持たない。

通常の対応は「公理を足す」「論理を拡張する」「より豊かな形式を使う」である。しかしそれは深いパターンを見落としている。そこで次の問いを立てる：

これらの隙間は欠陥ではなく「正準的に欠落した構造」であり、その構造は常にモナドなのではないか？

### 1.2 主張

**Theorem 1.1** (非形式的主張). 任意の形式体系  $N$ （論理・文法・計算）に対して、正準的な帰納モナド  $I_N$  が存在し、合成

$$\text{Lang}(N) := N \otimes I_N$$

は、実行意味・制御構造を備え、（穏やかな条件の下で）二諦随伴により停止性を持つプログラミング言語になる。

本稿はこの主張を厳密化し、10の異なる体系に渡って示す。

### 1.3 関連研究と位置付け

プログラミングにおけるモナドはよく研究されている (Moggi, Wadler, Spivey)。二諦随伴は新規である (Chiba, 2025)。本稿の貢献は、論理の隙間とモナドの対応を体系的に示し、二諦枠組みで統一する点にある。

## 2 二諦随伴と帰納モナド

### 2.1 二諦随伴の再確認

**Definition 2.1** (二諦随伴). 二諦随伴  $F_n \dashv G_{2^n+1}$  は次の 2-関手の対である：

- $F_n : C/D_{\text{fix}0} \rightarrow C^D$  (圧縮：メタレベル上昇、左緩)
- $G_{2^n+1} : C^D \rightarrow C/D_{\text{fix}0}$  (膠着：奇数周期による制約、右厳)

ここで  $C/D_{\text{fix}0}$  は日常的 (記述的) 2-スライス圏、 $C^D$  は Debt モナド  $D$  を持つ究極的 (計算的) EM 2-圏である。

随伴は次を保証する：

$$\text{Hom}_{C^D}(F_n(X), Y) \simeq \text{Hom}_{C/D_{\text{fix}0}}(X, G_{2^n+1}(Y))$$

日常的  $C/D_{\text{fix}0}$  は記述の領域であり、究極的  $C^D$  は実行の領域である。随伴により、記述力の不足は計算構造によって補われる。

### 2.2 帰納モナドの定義

**Definition 2.2** (帰納モナド). 論理または文法体系  $N$  (有限生成の構文または理論) に対し、 $N$ -帰納モナド  $I_N$  とは、次を満たす **Set** 上 (あるいは適切な圏上) のモナドである：

1. 対象写像が  $N$  の生成規則を自由構造の生成子として埋め込む。
2.  $\text{bind } \mu : I_N \circ I_N \Rightarrow I_N$  が  $N$  に欠けていた制御構造を実現する。
3.  $\text{return } \eta : \text{Id} \Rightarrow I_N$  が生データを自明なプログラムとして埋め込む。

**Kleisli 圏**  $\mathbb{K}(I_N)$  が実行可能言語をモデル化する：対象は型、射はプログラムである。

**Remark 2.1.** 帰納モナドは恣意的ではない。 $N$  が与えられると、どの計算的効果が欠けているかによって  $I_N$  は一意に定まる。この決定が後の対応定理の内容である。

### 2.3 二諦随伴との接続

随伴の役割は、 $F_n$  (メタレベル上昇) と  $G_{2^n+1}$  (膠着制約) の反復適用により、モナドの累積 Debt (未解消の複雑さ) を有限ステップでゼロに収束させる点にある。

**Lemma 2.1** (随伴によるモナド収束). 帰納モナド  $I_N$  が日常/究極圏と二諦随伴で結び付けられているとき、反復

$$F_n \circ G_{2^n+1} \circ F_{n+1} \circ G_{2^{n+1}+1} \circ \dots$$

によりノルム  $\|\text{Debt}\|$  は厳密に減少し、有限ステップでゼロに到達する。

**証明スケッチ：** 各合成  $F_n \circ G_{2^n+1}$  は構文深さ (K-複雑度) を圧縮しつつ、制約構造 ( $\ell$ -完全性) を強制する。ノルム  $\|\text{Debt}\| = K + \ell$  は整基底降下により減少する。

### 3 主要定理

**Theorem 3.1** (普遍モナド対応). 任意の形式体系  $N$  (論理・文法・計算) に対して、(同型を除いて) 一意な帰納モナド  $I_N^*$  が存在し、 $N \otimes I_N^*$  は  $N$  を実行可能にする最小の計算力を与える。さらにこのモナドは二諦随伴から回収できる：

$$I_N^* = \text{colim}_k G_{2^k+1}(F_k(N))$$

**Remark 3.1.** この定理は「欠けた構造は恣意に追加されるのではなく、日常/究極の双対性から現れる」という直観を形式化する。

**Theorem 3.2** (モナド-言語対応). 形式体系  $N$  と帰納モナド  $I_N$  について、次は同値である：

1. 言語  $\text{Lang}(N, I_N) := N \otimes I_N$  がチューリング完全 (または  $N$  の計算クラスに到達) である。
2.  $I_N$  が  $N$  の「欠落効果」をすべて捉える：実行の各ステップはモナド  $\text{bind } \mu$  に対応する。
3. ノルム  $\|\text{Debt}_N(I_N)\|$  が  $F_n \circ G_{2^n+1}$  の有限反復でゼロになる。

**Remark 3.2.** この定理は、あるモナド-体系の組が有効で別の組が失敗する理由を説明する (Peano + Free や Horn + List は成功、二値論理単体では単一モナドで完成しない)。

**Theorem 3.3** (二諦随伴による停止性).  $\text{Lang}(N, I_N)$  におけるプログラムが型付けされている (すなわち  $C/D_{\text{fix}0}$  の型に居住し、 $C^D$  の Debt 代数を尊重する) ならば、その実行は停止し、固定点  $D_{\text{fix}0}$  (空性の文脈における究極記述) に到達する。

**証明スケッチ：** 型付けされたプログラムは有限ノルムを持つ日常的射に対応する。随伴  $F_n \dashv G_{2^n+1}$  により、このノルムは各簡約ステップで厳密に減少し、 $\mathbb{N}$  の整基底降下により停止が保証される。

—

### 4 4つの典型例

#### 4.1 例1：ペアノ算術 + 自由モナド

体系：ペアノ算術  $\mathbb{P}$  ( $0 \in \mathbb{N}$ ,  $S : \mathbb{N} \rightarrow \mathbb{N}$ )。

隙間： $\mathbb{P}$  は「 $2 + 3 = 5$ 」を記述できるが、自由に計算できない。

欠落モナド： $I_{\mathbb{P}} = \text{Free}(\Sigma_+)$  ( $\Sigma_+$  は原始演算ごとの生成子を持つ関手)。

**Definition 4.1** (シグネチャ).

$$\Sigma_+(X) = \mathbb{N} + (X \times X)$$

(自然数定数、または部分式の対)。

**結果：**  $\mathbb{P} \otimes I_{\mathbb{P}}$  は項書換え系 (または純関数言語) となり：

- $\text{return}(n)$  が数値をモナドに持ち上げる。
- $\mu$  (bind) が構文木の合成を可能にする。
- 評価 (インタプリタ) が自由構造を再帰スキームで展開する。

**要点：** 自由モナドの再帰構造が、ペアノ算術に欠けていた「計算の中断と合成」を正確に補う。

## 4.2 例2：最小論理 + 継続モナド

体系：最小論理 ML ( $(\Rightarrow, \perp)$  のみ)。

隙間：ML は  $\neg p$  や脱出・例外を表現できない。

欠落モナド： $I_{\text{ML}} = \text{Cont}_R$  (答型  $R$  の継続モナド)。

**Definition 4.2** (継続モナド).

$$\text{Cont}_R(a) = (a \rightarrow R) \rightarrow R$$

$$\mu(m, k) = \lambda f. m(\lambda x. k(x)(f))$$

$$\eta(x) = \lambda f. f(x)$$

結果： $\text{ML} \otimes I_{\text{ML}}$  は call/cc を備えた古典論理となる：

- 含意  $p \Rightarrow q$  はプログラムとなる。
- 矛盾  $\perp$  は継続による脱出を引き起こす。
- 継続モナドにより否定が制御構造として「出現」する。

要点：古典論理は最小論理に「計算の中断（継続）」を加えたものとして現れる。

## 4.3 例3：文脈自由文法 + 状態モナド

体系：CFG 規則  $S \rightarrow \text{NP VP}$  など。

隙間：CFG は解析中の位置を追跡できない。

欠落モナド： $I_{\text{CFG}} = \text{State}_{\mathbb{N}}$  (入力位置  $\mathbb{N}$ )。

**Definition 4.3** (状態モナド).

$$\text{State}_S(a) = S \rightarrow (a \times S)$$

$$\mu(m, k) = \lambda s. \text{let } (x, s') = m(s) \text{ in } k(x)(s')$$

$$\eta(x) = \lambda s. (x, s)$$

結果： $\text{CFG} \otimes I_{\text{CFG}}$  は位置を自動的に伝播する再帰下降パーザになる：

- 各文法規則がパーザ関数になる。
- 状態が入力位置を保持する。
- bind が位置の受け渡しを自動化する。

要点：解析には「記憶（位置）」が必要であり、状態モナドがこれを型に持ち上げる。

## 4.4 例4：ホーン節 + リストモナド

体系：ホーン節  $p \leftarrow q_1, \dots, q_n$ 。

隙間：ホーン節は選択点やバックトラックを持たない。

欠落モナド： $I_{\text{Horn}} = \text{List}$  (非決定性)。

**Definition 4.4** (リストモナド).

$$\text{List}(a) = [a]$$

$$\mu(m) = \text{concat}(m)$$

$$\eta(x) = [x]$$

結果：  $\text{Horn} \otimes I_{\text{Horn}}$  は**Prolog**となる：単一化とバックトラック探索。

- 各ホーン節が選択肢になる。
- `bind`（連結）で全探索が行われる。
- 探索木はリストモナドの選択木に一致する。

要点： 論理プログラミングは関係的記述に探索を加えたものである。

## 5 モナド-体系対応表

| 体系 $N$               | 隙間 / 欠落    | 帰納モナド $I_N$                       | 結果言語                        |
|----------------------|------------|-----------------------------------|-----------------------------|
| 1. ペアノ算術             | 合成・評価      | $\text{Free}(\Sigma)$             | 算術言語                        |
| 2. 単純型付き $\lambda$   | 変数束縛・文脈    | $\text{Reader} + \text{Free}$     | メタプログラミング言語                 |
| 3. 最小論理              | 否定・制御      | $\text{Cont}_R$                   | 古典論理 + <code>call/cc</code> |
| 4. ホーン節              | バックトラック・選択 | $\text{List} / \text{Nondet}$     | Prolog                      |
| 5. 正規文法              | 列挙・解析      | $\text{Free}(\Sigma)$             | 正規表現エンジン                    |
| 6. 文脈自由文法            | 位置追跡       | $\text{State} + \text{Free}$      | 再帰下降パーザ                     |
| 7. 依存型理論             | 証明構成       | $\text{Free}(\text{proof trees})$ | 証明支援 / スクリプト言語              |
| 8. Presburger 算術     | 解探索        | $\text{Writer} + \text{List}$     | 制約ソルバ                       |
| 9. 非型付き $\lambda$ 計算 | エラー回復      | $\text{Maybe} / \text{Either}$    | 安全なインタプリタ                   |
| 10. 量子子なし一階論理        | 非決定探索      | $\text{List} / \text{Nondet}$     | 探索プログラミング                   |

Table 1: 10の典型体系と対応する帰納モナド

**Remark 5.1.** 各行は一つの発見である。形式体系と、それを実行可能に「完成」させる一意なモナドが対応している。これは偶然ではなく、二諦随伴により決まる。

## 6 メタレベル：なぜこれらのモナドか

### 6.1 最小付加の原理

**Principle 6.1.** 体系  $N$  に対する帰納モナド  $I_N$  とは、次の可換図式を成り立たせる最小の付加である：

$$\begin{array}{ccc}
 N & \xrightarrow{I_N} & N \otimes I_N \\
 \downarrow & & \downarrow \text{exec} \\
 \text{Syntax} & \longrightarrow & \text{Computation}
 \end{array}$$

これは随伴によって形式化される： $I_N$  は  $F_n$  による圧縮像として現れる。

## 7 メタレベル補完：二値論理から言語へ

**Theorem 7.1** (二値論理のメタ補完). 純粋な二値論理  $B$  は単一のモナドでは完成しないが、メタレベル上昇  $F_n$  は自動的に構造的な隙間を生成し、その隙間が正準的なモナド  $I_{B^{(n)}}$  を誘導する。

したがって合成言語

$$\text{Lang}(B^{(n)}) := B^{(n)} \otimes I_{B^{(n)}}$$

はチューリング完全となる。ここで  $B^{(n)} = F_n(B)$  であり、 $I_{B^{(n)}}$  は二諦随伴から生じる帰納モナドである。

言い換えれば、二値論理の不完全性は欠陥ではなく、メタレベル構造の必然性を指し示すものであり、これはまさに空性（*Śūnyatā*）の空間に対応する。

—

## 8 応用：LLM 推論を帰納モナドとして見る

LLM の推論ループは次のように記述できる：

1. トークン生成（return）。
2. 文脈整合性の検査（bind）。
3. 継続か停止か（モナド選択／効果）。

各ステップはモナド演算であり、停止条件は定理 3.3 の条件と一致する：推論鎖の未解消 Debt がゼロになると停止する。

**Corollary 8.1** (LLM の停止). 適切な  $N$  と  $I_N$  に対する  $\text{Lang}(N, I_N)$  のプログラムとして LLM の推論過程を見ると、それはノルムがゼロになるときにのみ停止する。

—

## 9 未解決問題

1. モナド自動発見：体系  $N$  から  $I_N$  を計算的に導出できるか？（ホモロジーや Tor 計算による可能性。）
2. 複合モナド体系：複数モナド（State + Exception など）が必要な場合の正規形はあるか？
3. 逆解析：言語  $L$  を  $N \otimes I_N$  に分解して最小体系  $N$  を回収できるか？
4. 計算複雑性：モナドの「階数」と結果言語のチューリング次数に関係はあるか？
5. 証明支援：定理 3.3 は依存型やホモトピー型理論に拡張できるか？

—

## 10 結論

形式体系と実行可能言語の間にある隙間は、恣意的に埋めるべき欠陥ではなく、**正準的に決まる構造**である。二諦随伴  $F_n \dashv G_{2^n+1}$  は、記述と実行が分断された領域ではなく、同一過程の双対的な側面であることを示す。

帰納モナドの原理は、再帰・制御・バックトラック・状態といった構成要素を一つの数学的枠組みに統合する。これは、**表現力は偶然ではなく、形式体系の正準的な隙間を埋めることによって出現する**という深い原理を示唆する。

—

## References

- [1] E. Moggi, “Notions of computation and monads,” **Information and Computation**, 93(1):55–92, 1991.
- [2] P. Wadler, “The essence of functional programming,” **Proc. POPL**, pp. 1–14, 1992.
- [3] S. Mac Lane, **Categories for the Working Mathematician**, Springer, 1971.
- [4] F. W. Lawvere, “Adjointness in foundations,” **Dialectica**, 23(3-4):281–296, 1969.
- [5] M. Chiba, “Two-truth adjunction and Alethic Complexity Integration Formalism,” 2025 (forthcoming).
- [6] M. Spivey, “A categorical approach to the semantics of name binding,” **Theoretical Computer Science**, 151(2):437–475, 1990.
- [7] G. D. Plotkin and J. Power, “Computational effects and operations,” **Proc. APPSEM II**, LNCS 3622, Springer, 2005.